

SA PUNE LAB inventory

- Purpose
- Hardware Details
- Proxmox UI login
- Current Network architecture overview
- Code of conduct
- Creating and Managing VMs
 - Using Available templates
 - Building and cloning via ISO images
 - Using cloudimage/qcow2 to boot a new VM
 - Making use of the SRIOV VFs on the hosts
 - Caveats
- Template creation
 - For VMs that are cloned with the starter template available in the setup:
- PCD environments
 - Adding proxmox VMs into PCD
 - Architecture
 - Hostnetwork configurations:
 - Applying Glance role
 - Applying Cinder role
- Use cases
 - Creating MaaS environment for local use
 - Architecture
 - Configuration

Purpose

This doc contains the list of hardware inventory that is managed and used by the SA team. Configure and use proxmox hypervisors for local testing and validation

Hardware Details

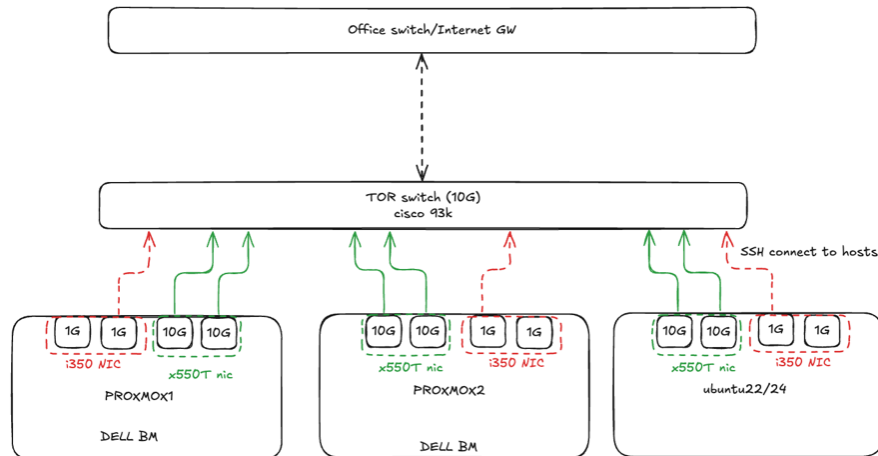
Server Make n Model	iDRAC IP	Host IP	Comments
Dell Power Edge R640	10.96.12.147	10.96.6.240	Proxmox Host
Dell Power Edge R640	10.96.1.153	10.96.14.184	Proxmox Host
Dell Power Edge R640	10.96.4.57	10.96.14.185	Host for MaaS and Tests

CPU	Cores	RAM	Storage
Intel(R) Xeon(R) Gold 6138 CPU @ 2.00GHz	2 X 20	256 GB	2 X 1 TB

Proxmox UI login

<https://10.96.6.240:8006/>

Current Network architecture overview



- The 10G ports are individually consumed on every host.
- 2 hosts dedicatedly run as Promox hypervisor
- 1 host is standalone baremetal that can be used for PCD hypervisor testing or for MaaS test operations.

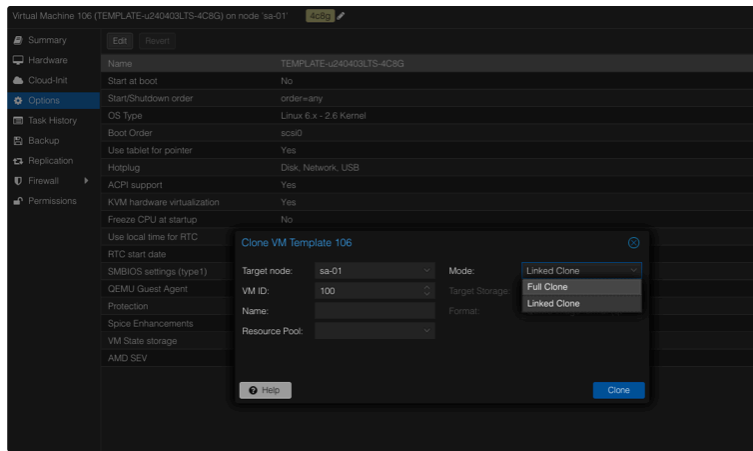
Code of conduct

- There are finite number resources available within the infrastructure hence it is encouraged to shutdown or delete any VMs that are not actively used.
- Do not delete or create linked clone of resources starting with **TEMPLATE_**. This is reserved for only creating templates of VMs that can be consume by all in the team.
- Any new templates creation for testing, ensure have it named as **yourName-os_name_flavorsize** for example **Arvind_U24LTS_8C16G**.
- **vmb1** is the default bridge on all the hypervisor. Ensure this bridge is not disrupted during testing.
- Do not migrated VMs that has VF or other physical resource pinned. It may fail because the same PCI device may or may not be available in the destination resource.
- Create a **Separate cluster** with your name as prefix in the PCD and try primarily using it for testing unless there are any specific other needs to use any other existing cluster in the PCD UI.
- **Do not create Bridges on top of SRIOV VFs**. It will not work as intended and will disturb the overall connection to the environment in severe conditions

Creating and Managing VMs

Using Available templates

- Login to UI and create a VM by cloning the template with name **TEMPLATE-u240403LTS-4C8G**. In the clone page, ensure to select a full clone.



- Wait for the moment for it to be ready. It will create a VM with 4core, 8GB of ram and a 130GB of volume that will be the latest ubuntu24.04 LTS release.
- It will be default attached to `vmbri` bridge and will acquire an IP from the external network
- The default login user is `ubuntu`.

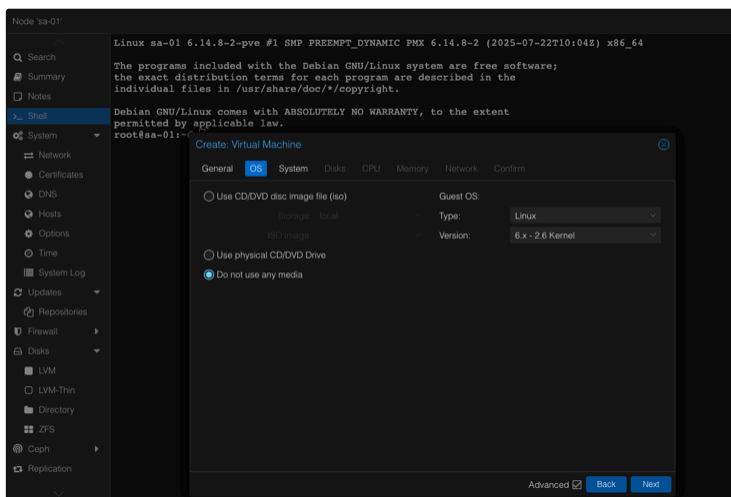
Building and cloning via ISO images

- A volume can be created by deploying a fresh VM by attaching it with a Virtual ISO and cloning or making a fresh template out of it.

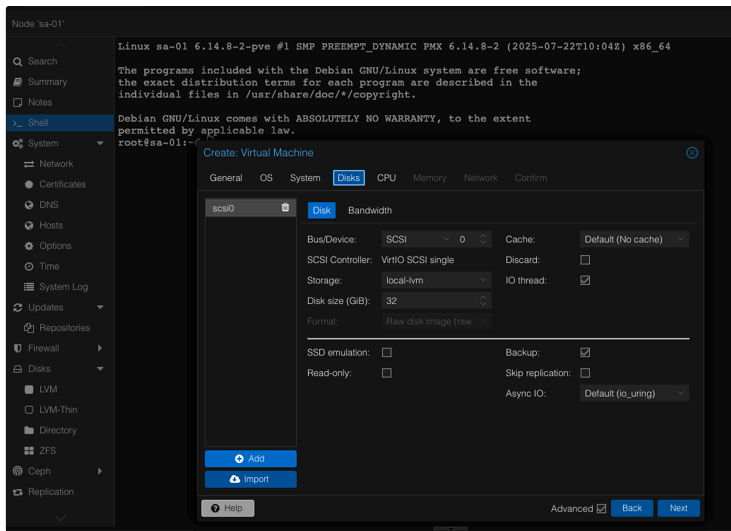
Using cloudimage/qcow2 to boot a new VM

This is not the default method within proxmox ecosystem, being said here is the workflow on creating VM with cloud image similar to creating volume based instance with PCD.

- Create a VM by selecting do not use any media option for the OS tab in the UI:



- Remove the attached disks the will be visible by default in the disks section:



- Set the required CPU and memory resource and create the VM. **Do not boot the VM yet.**
- Make a note of the VM ID that would be visible in the UI:

108 (VM 108)

- Go into the hypervisor SSH terminal and download the required cloud-image as needed, for example to download the latest ubuntu24 the wget command would be like below:

```
1 wget https://cloud-images.ubuntu.com/noble/20250704/noble-server-cloudimg-amd64.img
```

- Create a backup of the downloaded image:

```
1 cp noble-server-cloudimg-amd64.img noble-server-cloudimg-amd64-backup.img
```

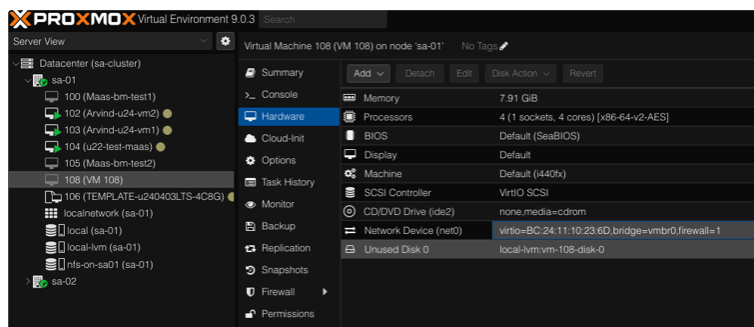
- Resize the image to the required size:

```
1 qemu-img resize noble-server-cloudimg-amd64.img 120G
```

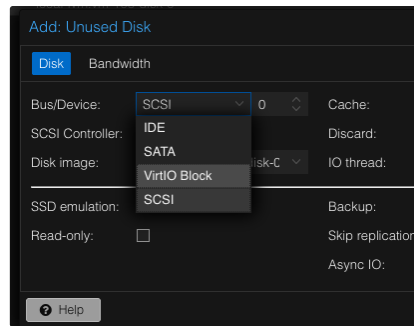
- Copy the resized disk to the volume store with below command:

```
1 qm importdisk <vm_id> noble-server-cloudimg-amd64.img <storage_name>
2 example:
3 qm importdisk 108 noble-server-cloudimg-amd64.img
```

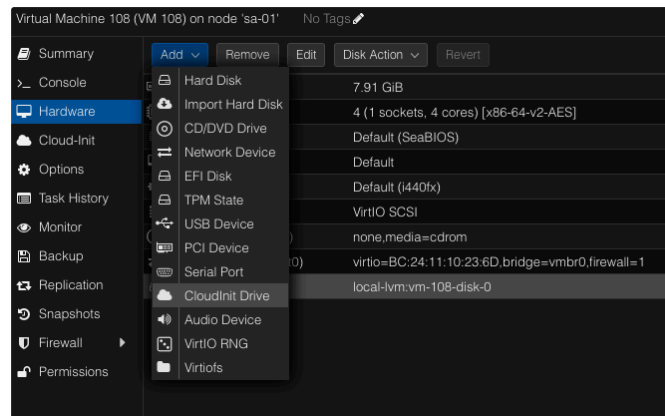
- At this point, after the above step, the disk will be attached as unused disk under the hardware section in the UI:



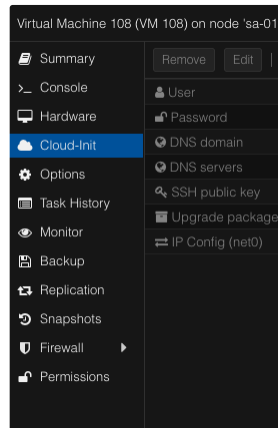
- Select the **unused Disk 0** and select the **edit** option, select the **virtio-block** option and save:



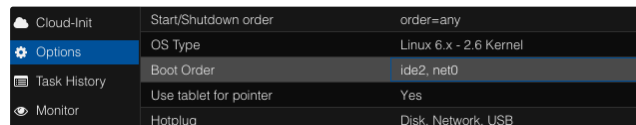
- Add a cloud-init drive to the host:



- Select the cloud-init under the summary section which will be now active:



- Select the password to edit and set the needed password.
- Select the Options under the Summary and edit the boot order to ensure the newly added boot volume is visible and is set to top priority (it can be dragged to the top in the UI)



- After this point the VM can be powered on and it will boot from the disk attached.
- Post getting into console, run the `df` or `df -hT` command to check the partition, it would usually be something like this **if it is downloaded cloud image**:

```

Filesystem      1K-blocks    Used Available Use% Mounted on
tmpfs           201500         1004    200496   1% /run
/dev/sda1       2466612 1643952    806276  68% /
tmpfs           1007496         0    1007496   0% /dev/shm
tmpfs           5120          0      5120    0% /run/lock
/dev/sda16      901520     62372    776020   8% /boot
/dev/sda15     106832     6246    100586   6% /boot/efi
tmpfs           201496         12    201484   1% /run/user/1000

```

- **IF and only IF this is the layout noted**, run the parted command and resize the sda1 to use the whole disk:

```
1 parted /dev/sda
```

- the print command in the parted should be something like below:

```

(parted) print
Model: QEMU QEMU HARDDISK (scsi)
Disk /dev/sda: 14.5GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name  Flags
  14     1049kB  5243kB  4194kB                bios_grub
  15     5243kB  116MB   111MB    fat32         boot, esp
  16     116MB   1074MB  957MB    ext4          bls_boot
  1      1075MB  3758MB  2683MB    ext4

```

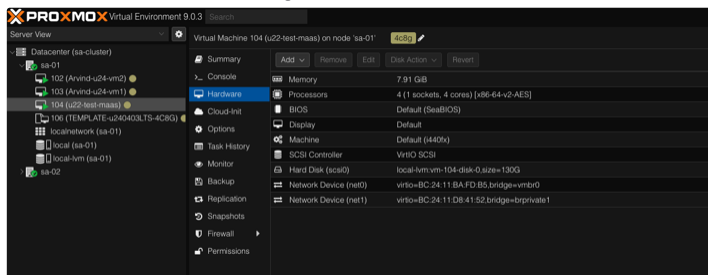
- Run the `resizepart` on the `partition number 1` in the disk:

```
1 resizepart 1 100%
```

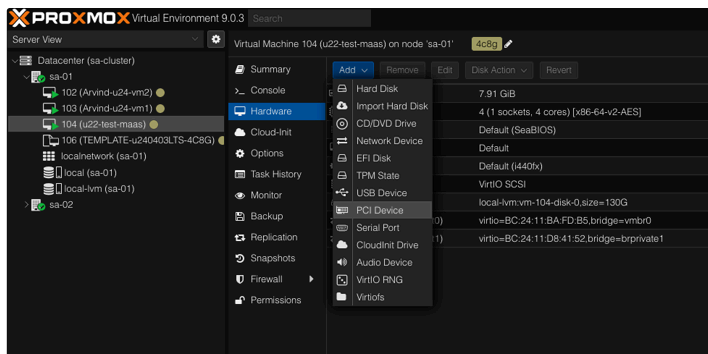
This should ensure the whole disk is consumed and the VM is good to be used for further operations.

Making use of the SRIOV VFs on the hosts

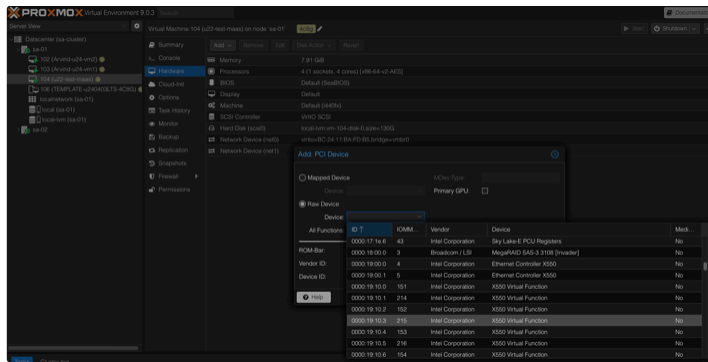
- Both proxmox hosts have 2 10G NICs and each NIC has 63 VFs that can be passthrough to VM for any testing effectively having a line rate communication for the VMs.
- Once the VFs is consumed by a VM it will not be visible to the underlying hypervisor.
- From the UI VFs can be attached to a running instance:
 - Click the instance and and go to hardware tabs:



- Select the PCI device in the hardware selection:



- Select the raw device in the PCI window and look for the items that reads `x550 virtual functions`:



- Already consumed ones will not be listed here.
- Proceed to attach it to the VM
- It can now be used like any other network device.

Caveats

- There is no specific way to directly tell to which PF a specific VF would belong for the SRIOV nics in the UI.
- The command line way to identify will be help (Steps to do that will be updated shortly in the doc)

Template creation

To create a template that will be used by everyone like an golden image, One would need to ensure:

- There are no reference to any older stray entries in the original VMs.
- Remove any SSH Keys or other details from the VM
- Install the [qemu-guest-agent](#) packages:

```
1 apt-get install qemu-guest-agent
```

- There is a know issue with [Proxmox VMs getting duplicate IPs when cloned from templates](#). To Mitigate this, **For fresh template creation**, run the following steps:
 - Inside your VM under `/opt/scripts/` directory (create this directory if not present) create a shell script with name `oneshot-uuid_reapply.sh` and contain the following:

```
1 /opt/scripts/
2 #!/usr/bin/env bash
3 set -eou pipefail
4
5 MARKER="/var/lib/firstboot/run"
6 SERVICE="firstboot-reset.service"
7 SERVICE2="firstboot-reset.path"
8 WAIT_SECONDS="${WAIT_SECONDS:-5}"
9
10 log(){ echo "[firstboot-reset] $*"; }
11
12 # Only run once if marker exists
13 if [[ ! -f "$MARKER" ]]; then
14   log "Marker not present; exiting."
15   exit 0
16 fi
17
18 log "Stopping network managers if present..."
19 if systemctl list-unit-files | grep -q '^NetworkManager.service'; then
20   systemctl stop NetworkManager || true
21 fi
22 if systemctl list-unit-files | grep -q '^systemd-networkd.service'; then
23   systemctl stop systemd-networkd || true
24 fi
25
26 log "Bringing all non-loopback interfaces down..."
27 for devpath in /sys/class/net/*; do
28   dev="$(basename "$devpath")"
29   [[ "$dev" == "lo" ]] && continue
30   ip link set dev "$dev" down || true
31 done
32
```

```

33 log "Resetting machine-id and DBus machine-id..."
34 rm -f /etc/machine-id
35 dbus-uuidgen --ensure=/etc/machine-id
36 rm -f /var/lib/dbus/machine-id || true
37 dbus-uuidgen --ensure
38
39 # Print the newly generated machine-id from /etc/machine-id
40 MID="$(cat /etc/machine-id 2>/dev/null || true)"
41 log "New machine-id: ${MID}"
42
43 # Wait before disabling and rebooting
44 log "Waiting ${WAIT_SECONDS}s before disabling service and rebooting..."
45 sleep "${WAIT_SECONDS}"
46
47 log "Disabling service and removing marker so it never runs again..."
48 systemctl disable "$SERVICE" || true
49 systemctl disable "$SERVICE2" || true
50 rm -f "$MARKER"
51
52 sync
53 log "Rebooting now..."
54 reboot

```

- Create A systemd **.service** file and a **.path** files under the **/etc/systemd/system/** location as below which will point to the above script:

```

1 cat /etc/systemd/system/firstboot-reset.service
2 [Unit]
3 Description=First-boot one-time reset (network down + machine-id)
4 DefaultDependencies=yes
5 Before=NetworkManager.service systemd-networkd.service
6 ConditionPathExists=/var/lib/firstboot/run
7
8 [Service]
9 Type=oneshot
10 ExecStart=/opt/scripts/oneshot-uuid_reapply.sh
11 TimeoutStartSec=300
12
13 [Install]
14 WantedBy=multi-user.target
15
16 ---
17
18 cat /etc/systemd/system/firstboot-reset.path
19 [Unit]
20 Description=Trigger firstboot-reset when marker exists
21
22 [Path]
23 PathExists=/var/lib/firstboot/run
24 Unit=firstboot-reset.service
25
26 [Install]
27 WantedBy=multi-user.target
28

```

- re-load the **systemd** service and shutdown the VM:

```

1 systemctl daemon-reload
2 systemctl enable firstboot-reset.path firstboot-reset.service && shutdown -h now

```

For VMs that are cloned with the starter template available in the setup:

- Ensure you have the required configurations/settings in place.
- Create the **/var/lib/firstboot/run** file
- Enable the **firstboot-reset** script and shutdown the VM:

```

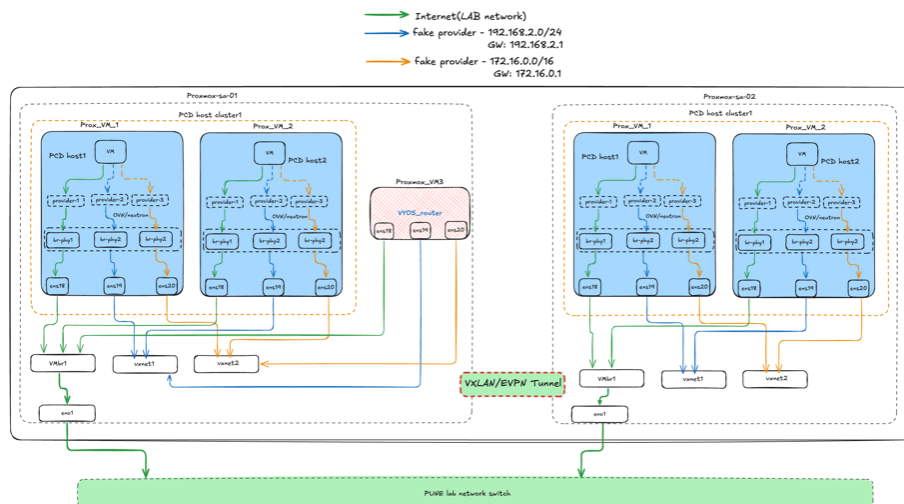
1 systemctl enable firstboot-reset.path firstboot-reset.service && shutdown -h now

```


		GW: 192.168.2.1 (on Vyos)
172.16.0.0/16	vxnet2 vxlan bridge	Fake provider2 172.16.0.0/16 GW: 172.16.0.1(on Vyos)
NO IP - native VLAN 400	eno2 interface	Used mainly for MaaS testing

NOTE: When using more than one provider to connect the **PCDV** VMs at the time of creation, **you would need to delete all the default route and leave one that is needed. Leaving multiple default route can lead to PCD vm being unreachable.**

Architecture



- For Single region deployment, to ensure better management of storage and glance resources, it is **recommended to main separate host clusters** for every user or use-case if needed. Preferable the host cluster should carry an easily identifiable name.

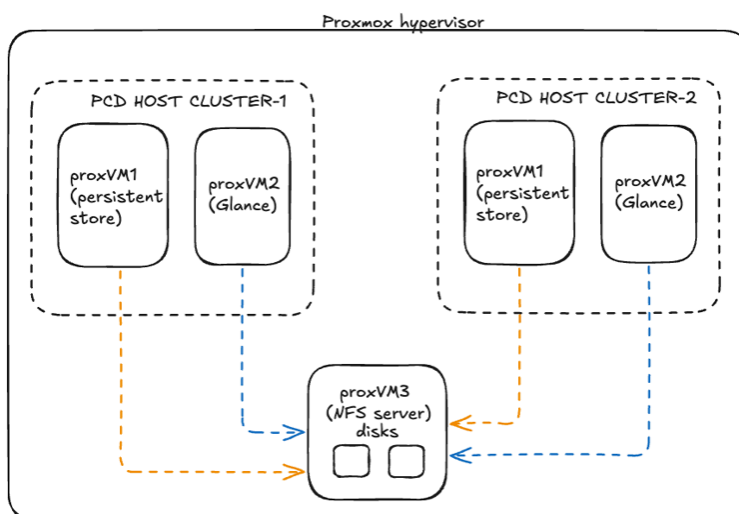
Hostnetwork configurations:

- It is recommended to create a separate host network config profile as per the host cluster and VM requirements.
- Some times the VM may have a different interface name due any frequent change or improper enumeration after reboot (not frequent but noticed) which will not match the host config in the blue print, in those case the VM level **netplan** can be used to update the device name matching to the profile with **match** and **set-name** options for example here if the **macaddress** matches the provided one the interface will be renamed to **ens19** :

```

1  ens19:
2      match:
3          macaddress: bc:24:11:76:68:7a
4      set-name: ens19
5      dhcp4: false
6      addresses:
7          - 192.168.2.11/24

```



- To ensure the image is common for all the clusters, a single NFS server is created in the SA-01 -proxmox host which needs to be externally mounted on the glance specific PCD host. Details of the NFS server:
 - IP address: 10.96.11.160
 - mount point: `/mnt/nfs_share/glance`
- After applying the glance role to the pcd host VM**, mount the nfs share persistently on the host under the default glance directory `/var/opt/imagelibrary/data`:

```
1 mount -t nfs 10.96.11.160:/mnt/nfs_share/glance /var/opt/imagelibrary/data
```

- Add the entry in the `etc/fstab` and restart the glance service

```
1 cat /etc/fstab
2 10.96.11.160:/mnt/nfs_share/glance /var/opt/imagelibrary/data nfs defaults 0 0
3
4 systemctl restart pf9-glance-api.service
```

Applying Cinder role

- Cinder role can either **specific to host cluster** or set to use the common nfs as backend if needed.
- For **cluster specific cinder backend**, ensure to add the configuration in the cluster blueprint **section with identifiable name** like:

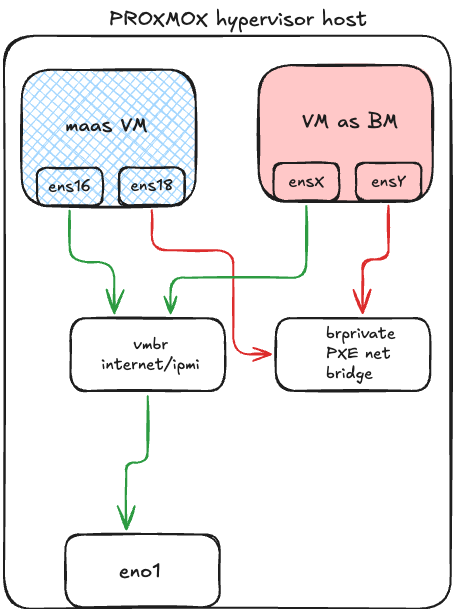
The screenshot shows the 'Storage Volume Types' section in the OpenStack dashboard. Under 'Volume Type', 'proxsa_arvind_cinderLVM' is selected. The 'Volume Backend Configurations' table shows a configuration for 'sa_arvind_cinder' with a driver of 'LVM'. There is an 'Add Configuration' button with a plus sign.

- While Assigning the storage role select the specific volume as set in blueprint to match the **host cluster** in use
- One must ensure to deploy the VMs with the respective volume backend depending on the host cluster in use.**

Use cases

Creating MaaS environment for local use

Architecture



Configuration

- Setup up a MaaS server using the any suitable method of deploying a ubuntu 24 vm first with one interface wired to `vmbr` and another to `brprivate` bridge
- Create blank VMs with no OS and attach them similarly to the same bridge.
- The first interface of MaaS will have an IP to the network the second interface connect to `brprivate` can be used to configure any IP range as its entirely private with no external network access.
- From the MaaS UI start adding the Machine with following power configuration:

Configuration	Value
Power type	Proxmox
Proxmox host name or IP	<IP address of hypervisor carrying blank VMs>
Proxmox user name	root@pam
proxmox password	somepassword
Node-ID	ID of the blank VM visible in the UI
Verify SSL connection	NO

- Save and start the introspection/commissioning of those VMs as baremetals